

PROJECT REPORT

Project no: 11

Project title:

Smart Inventory Management System for Construction Sites

Submitted by:

Arundepak S – 24MER004

Deepak P – 24MER006

Dharanidharan R S – 24MER007

Dinesh K – 24MER009

Vishnuvarthan P – 24MER063

Smart Inventory Management System for Construction Sites

1. Project Overview

The Smart Inventory Management System is an IoT-based solution designed to automate the tracking of construction materials — specifically cement bags and steel rods — as they move in and out of a construction site store. The system uses RFID (Radio-Frequency Identification) technology to identify tagged materials, an ESP32 microcontroller to process and act on the scan events, an OLED display for on-site visual feedback, a buzzer for audible confirmation, and Adafruit IO as a cloud platform for real-time inventory dashboards and historical movement logs.

Every time a registered RFID tag is scanned, the system automatically determines whether it represents a stock-in or stock-out event, updates the corresponding material quantity, displays the movement on the OLED screen, sounds a distinct buzzer pattern, and pushes the updated quantity along with a timestamped log entry to the cloud. This removes the need for manual register-based tracking and gives site managers a live, remotely accessible view of material stock levels.

2. Problem Statement

Construction sites typically manage material inventory (cement, steel, sand, bricks, etc.) using manual registers or spreadsheets updated by site supervisors. This traditional approach suffers from several well-known issues:

- Manual entry is time-consuming, error-prone, and dependent on worker discipline.
- Discrepancies between recorded and actual stock lead to material pilferage or unnoticed shortages.
- Site managers and procurement teams lack real-time visibility into material consumption, causing delayed reordering and project stoppages.
- There is no digital audit trail of who moved what material and when, making theft or misuse difficult to trace.
- Paper-based logs are difficult to consolidate across multiple sites or share with stakeholders remotely.

There is therefore a need for an automated, low-cost, wireless system that can continuously track material movement, provide immediate on-site confirmation, and make inventory data remotely accessible for proactive decision-making.

3. Proposed Solution

The proposed system uses an ESP32 as the central controller, interfaced with an MFRC522 RFID reader, an OLED display, and a buzzer. Key features of the solution:

- Unique RFID tag assigned to each material batch/category (Cement Bags and Steel Rods in this prototype).
- Automatic classification of each scan as STOCK IN (+10) or STOCK OUT (−5) by alternating the material's last movement state.

- On-site OLED display for immediate confirmation of material name, movement type, and updated quantity.
- Distinct buzzer patterns for stock-in, stock-out, and unregistered/unknown tag events.
- Wireless data upload to the Adafruit IO cloud platform on every scan, updating dedicated feeds for each material.
- Remote dashboard on Adafruit IO with live gauges and a chronological movement log, viewable from anywhere.
- Modular, low-cost hardware suitable for scaling to additional material types and multiple site entry points.

4. System Architecture

The system follows a three-layer IoT architecture:

- Perception Layer: MFRC522 RFID reader captures the unique ID (UID) of a scanned material tag.
- Processing / Edge Layer: The ESP32 matches the UID against a registered material database, determines the movement type (IN/OUT), updates the local quantity, and drives the OLED display and buzzer.
- Network & Cloud Layer: Data is sent over Wi-Fi to the Adafruit IO platform, where it is stored in dedicated feeds and visualized on live gauges and an activity log.

Data flow: RFID Tag → MFRC522 Reader → ESP32 (processing & alert logic) → OLED/Buzzer (local feedback) → Wi-Fi → Adafruit IO Cloud (remote monitoring).

5. Circuit Table

Component	Pin	ESP32 Pin	Function
MFRC522 RFID	SDA / SS	GPIO 5	Chip select
MFRC522 RFID	RST	GPIO 27	Reset
MFRC522 RFID	SCK / MOSI / MISO	GPIO 18 / 23 / 19	SPI bus
SSD1306 OLED	SDA	GPIO 21	I2C data
SSD1306 OLED	SCL	GPIO 22	I2C clock
Buzzer	Signal (+)	GPIO 25	Audible alert
All Modules	VCC / GND	3V3 / G	Power

6. Circuit Design

The MFRC522 RFID reader, OLED display, and buzzer are wired to the ESP32 as per the circuit table above, with common power (3.3V) and ground rails distributed via the breadboard. The RFID module communicates over SPI using GPIO5 (SS), GPIO27 (RST), and the hardware SPI pins (GPIO18/23/19), the OLED communicates over I2C using GPIO21 (SDA) and GPIO22 (SCL), and the buzzer is driven directly from GPIO25.

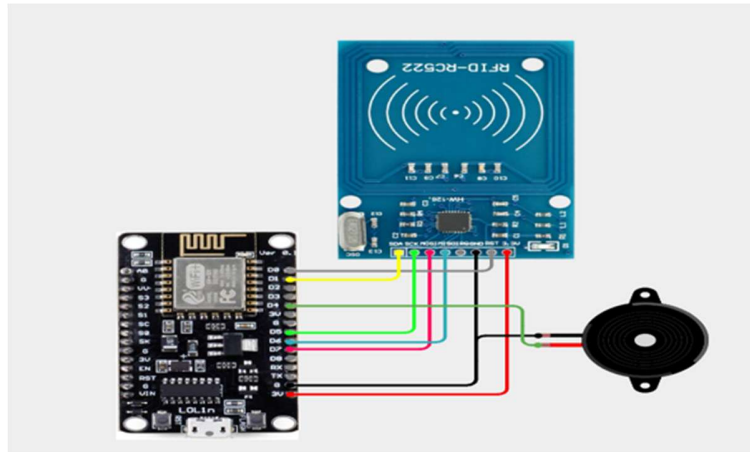


Figure 1: Wiring diagram of ESP32/NodeMCU with MFRC522 RFID reader and buzzer

7. Algorithm

Step-by-step operating logic implemented in firmware:

- Initialize Serial, buzzer pin, I2C bus/OLED, and RFID reader (SPI); display a startup message.
- Connect to the configured Wi-Fi network and Adafruit IO; wait until connection status is AIO_CONNECTED.
- Push initial inventory quantities and a 'System started' message to the cloud feeds; display the idle 'READY' screen.
- In the main loop, continuously call io.run() to keep the cloud connection alive.
- Poll the RFID reader for a new card; if none is present, skip to the next loop iteration.
- On a valid card read, build the UID string from the card's byte data and search the local inventory array for a match.
- If the UID is not found: display 'UNKNOWN TAG', sound a long buzzer alert, and log the unknown scan to the cloud.
- If the UID is found: toggle the material's last movement state to determine STOCK IN or STOCK OUT.
- Update the quantity (+10 for IN, -5 for OUT, clamped at 0) and trigger the corresponding short (IN) or double (OUT) beep.
Update the OLED with the material name, movement type, and new quantity; push the update to the material's Adafruit IO feed and a formatted message to the movement-log feed.
- Halt the RFID card session, wait briefly, return the OLED to the idle screen, and repeat from Step 4.

8. Coding

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include "AdafruitIO_WiFi.h"

#define WIFI_SSID  "YOUR_WIFI_SSID"
#define WIFI_PASS  "YOUR_WIFI_PASSWORD"
#define IO_USERNAME "YOUR_ADAFRUIT_IO_USERNAME"
#define IO_KEY      "YOUR_ADAFRUIT_IO_KEY"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

AdafruitIO_Feed *cementFeed = io.feed("cement-bags");
AdafruitIO_Feed *steelFeed  = io.feed("steel-rods");
AdafruitIO_Feed *logFeed    = io.feed("movement-log");

#define RFID_SS  5
#define RFID_RST 27

#define OLED_SDA 21
#define OLED_SCL 22

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define BUZZER_PIN 25

MFRC522 rfid(RFID_SS, RFID_RST);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

struct Material {
  String uid;
  String name;
```

```

int quantity;
const char* unit;
AdafruitIO_Feed* feed;
};

Material inventory[] = {
    { "BA 8A 81 32", "Cement Bags", 50, "bags", nullptr },
    { "C1 77 5C 68", "Steel Rods", 30, "rods", nullptr }
};

const int NUM_MATERIALS = sizeof(inventory) / sizeof(inventory[0]);
int findMaterial(String uid) {
    for (int i = 0; i < NUM_MATERIALS; i++) {
        if (inventory[i].uid == uid) return i;
    }
    return -1;
}

void buzzerOff() {
    noTone(BUZZER_PIN);
    digitalWrite(BUZZER_PIN, HIGH);
}

void beepIn() {
    digitalWrite(BUZZER_PIN, LOW);
    delay(150);
    buzzerOff();
}

void beepOut() {
    for (int i = 0; i < 2; i++) {
        digitalWrite(BUZZER_PIN, LOW);
        delay(130);
        buzzerOff();
        delay(80);
    }
}

```

```
void beepUnknown() {  
    digitalWrite(BUZZER_PIN, LOW);  
    delay(550);  
    buzzerOff();  
}
```

```
void showIdleScreen() {  
    display.clearDisplay();  
    display.setTextColor(SSD1306_WHITE);  
    display.setTextSize(1);  
    display.setCursor(0, 0);  
    display.println("SMART INVENTORY");  
    display.drawLine(0, 10, 127, 10, SSD1306_WHITE);  
    display.setTextSize(2);  
    display.setCursor(0, 22);  
    display.println("READY");  
    display.setTextSize(1);  
    display.setCursor(0, 50);  
    display.println("Scan RFID Tag");  
    display.display();  
}
```

```
void showMovement(String matName, int qty, const char* unit, bool isIn) {  
    display.clearDisplay();  
    display.setTextColor(SSD1306_WHITE);  
  
    display.setTextSize(1);  
    display.setCursor(0, 0);  
    display.println("SMART INVENTORY");  
    display.drawLine(0, 10, 127, 10, SSD1306_WHITE);  
  
    display.setTextSize(1);  
    display.setCursor(0, 15);  
    display.println(matName);  
  
    display.setTextSize(2);
```

```

display.setCursor(0, 28);
display.println(isIn ? "STOCK IN" : "STOCK OUT");

display.setTextSize(1);
display.setCursor(0, 50);
display.print("Qty: ");
display.print(qty);
display.print(" ");
display.println(unit);

display.display();
}

void showUnknownTag(String uid) {
display.clearDisplay();
display.setTextColor(SSD1306_WHITE);
display.setTextSize(1);
display.setCursor(0, 0);
display.println("SMART INVENTORY");
display.drawLine(0, 10, 127, 10, SSD1306_WHITE);
display.setTextSize(1);
display.setCursor(0, 20);
display.println("UNKNOWN TAG!");
display.setCursor(0, 35);
display.println(uid);
display.setCursor(0, 50);
display.println("Not in database");
display.display();
}

void setup() {
Serial.begin(115200);
delay(1500);

Serial.println();
Serial.println("=====");
Serial.println(" SMART INVENTORY SYSTEM - CONSTRUCTION SITE");

```



```
Serial.println("=====");

pinMode(BUZZER_PIN, OUTPUT);
buzzerOff();

Wire.begin(OLED_SDA, OLED_SCL);

if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
  Serial.println("OLED NOT FOUND! Check wiring/address.");
} else {
  Serial.println("OLED READY");
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("SMART INVENTORY");
  display.setCursor(0, 25);
  display.println("Connecting WiFi...");
  display.display();
}

SPI.begin(18, 19, 23, 5);
rfid.PCD_Init();
delay(100);
Serial.println("RFID READER READY");

inventory[0].feed = cementFeed;
inventory[1].feed = steelFeed;

Serial.print("Connecting to Adafruit IO");
io.connect();

while (io.status() < AIO_CONNECTED) {
  Serial.print(".");
  delay(500);
}
Serial.println();
```

```
Serial.println(io.statusText());
```

```
Serial.println("---- CURRENT INVENTORY ----");
```

```
for (int i = 0; i < NUM_MATERIALS; i++) {
```

```
    Serial.print(inventory[i].name);
```

```
    Serial.print(": ");
```

```
    Serial.print(inventory[i].quantity);
```

```
    Serial.println(inventory[i].unit);
```

```
}
```

```
Serial.println("-----");
```

```
cementFeed->save(inventory[0].quantity);
```

```
steelFeed->save(inventory[1].quantity);
```

```
logFeed->save("System started");
```

```
showIdleScreen();
```

```
beepIn();
```

```
}
```

```
bool lastWasIn[2] = { true, true };
```

```
const int IN_STEP = 10;
```

```
const int OUT_STEP = 5;
```

```
void loop() {
```

```
    io.run();
```

```
    if (!rfid.PICC_IsNewCardPresent()) return;
```

```
    if (!rfid.PICC_ReadCardSerial()) return;
```

```
    String uid = "";
```

```
    for (byte i = 0; i < rfid.uid.size; i++) {
```

```
        if (rfid.uid.uidByte[i] < 0x10) uid += "0";
```

```
        uid += String(rfid.uid.uidByte[i], HEX);
```

```

    if (i < rfid.uid.size - 1) uid += " ";
}
uid.toUpperCase();

Serial.println("=====");
Serial.print("UID: ");
Serial.println(uid);

int idx = findMaterial(uid);

if (idx == -1) {
    Serial.println("Result: UNKNOWN TAG - not registered");
    showUnknownTag(uid);
    beepUnknown();
    logFeed->save("Unknown tag scanned: " + uid);
} else {
    bool isIn = !lastWasIn[idx];
    lastWasIn[idx] = isIn;

    if (isIn) {
        inventory[idx].quantity += IN_STEP;
        Serial.print("Movement: STOCK IN (+");
        Serial.print(IN_STEP);
        Serial.println(")");
        beepIn();
    } else {
        inventory[idx].quantity -= OUT_STEP;
        if (inventory[idx].quantity < 0) inventory[idx].quantity = 0;
        Serial.print("Movement: STOCK OUT (-");
        Serial.print(OUT_STEP);
        Serial.println(")");
        beepOut();
    }

    Serial.print("Material: ");
    Serial.println(inventory[idx].name);
    Serial.print("New Quantity: ");

```

```
Serial.print(inventory[idx].quantity);
Serial.println(inventory[idx].unit);

showMovement(inventory[idx].name, inventory[idx].quantity,
             inventory[idx].unit, isIn);

inventory[idx].feed->save(inventory[idx].quantity);

String logMsg = inventory[idx].name + " " +
               (isIn ? "STOCK IN (+)" : "STOCK OUT (-)") +
               String(isIn ? IN_STEP : OUT_STEP) + ") -> " +
               String(inventory[idx].quantity) + " " + inventory[idx].unit;
logFeed->save(logMsg);
}

Serial.println("=====");
Serial.println();

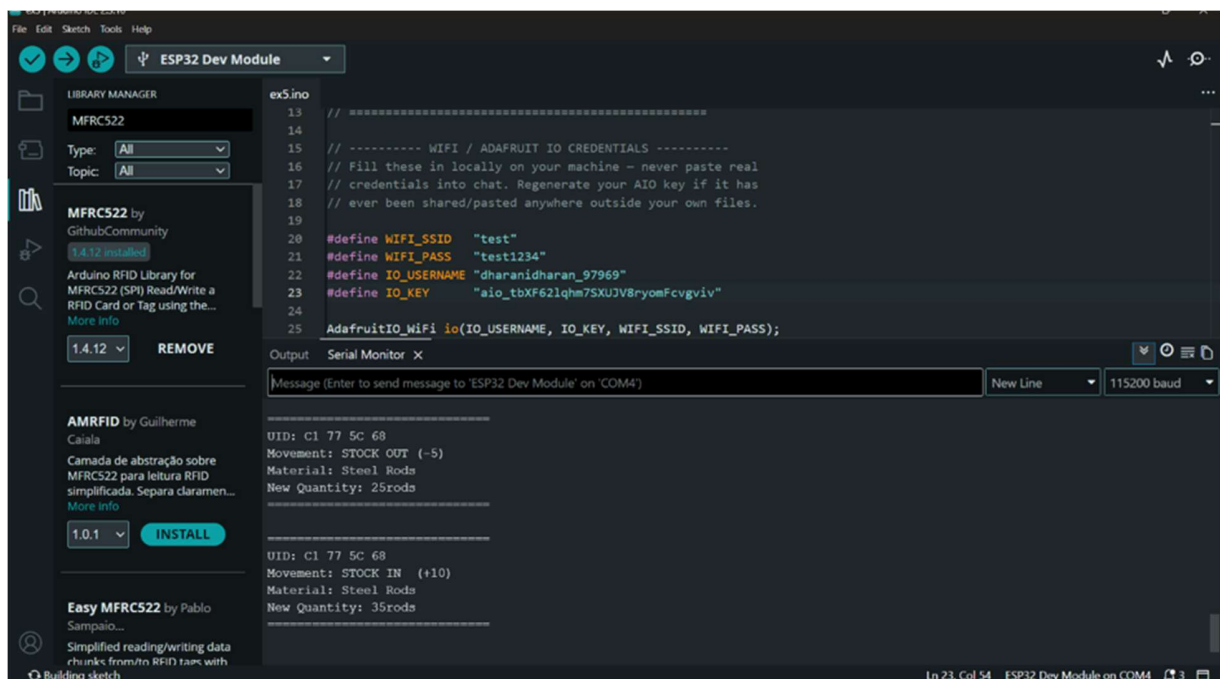
rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();

delay(2000);
showIdleScreen();
delay(500);
}
```

9. System Working

On power-up, the ESP32 initializes all peripherals and connects to the configured Wi-Fi network and Adafruit IO, confirming readiness on the OLED. During normal operation, the operator scans a registered RFID tag whenever a material batch enters or leaves the store. The system automatically alternates each scan between STOCK IN and STOCK OUT for that material, instantly updating the OLED display and giving audible confirmation via the buzzer: a single short beep for stock-in, a double short beep for stock-out, and a long 550 ms beep for an unregistered tag.

Simultaneously, the updated quantity is pushed to the corresponding Adafruit IO feed, and a human-readable log entry (material, movement, new quantity) is appended to the movement-log feed. The Adafruit IO dashboard displays two live gauges — one for Cement Bags and one for Steel Rods — along with a scrolling 'Recent Activity' panel showing the timestamped movement history, closing the loop between physical scanning, local alerting, and remote/cloud-based monitoring.



The screenshot shows the Arduino IDE interface with the ESP32 Dev Module selected. The Library Manager on the left shows the MFRC522 library installed. The main editor displays the code for the ESP32, which includes WiFi credentials and the AdafruitIO_WiFi library. The Serial Monitor at the bottom shows the output of the program, displaying the UID, movement type, material, and new quantity for two events: a stock-out and a stock-in.

```
13 // =====
14 // ----- WIFI / ADAFRUIT IO CREDENTIALS -----
15 // Fill these in locally on your machine - never paste real
16 // credentials into chat. Regenerate your AIO key if it has
17 // ever been shared/pasted anywhere outside your own files.
18
19
20 #define WIFI_SSID "test"
21 #define WIFI_PASS "test1234"
22 #define IO_USERNAME "dharanidharan_97969"
23 #define IO_KEY "aio_tbf62lqhm7SXUJv8ryomFcvgviv"
24
25 AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
```

Output Serial Monitor x

Message (Enter to send message to 'ESP32 Dev Module' on 'COM4') New Line 115200 baud

UID: c1 77 5c 68
Movement: STOCK OUT (-5)
Material: Steel Rods
New Quantity: 25rods

UID: c1 77 5c 68
Movement: STOCK IN (+10)
Material: Steel Rods
New Quantity: 35rods

Figure 4: Arduino IDE — Serial Monitor showing Cement Bags stock-out and stock-in events

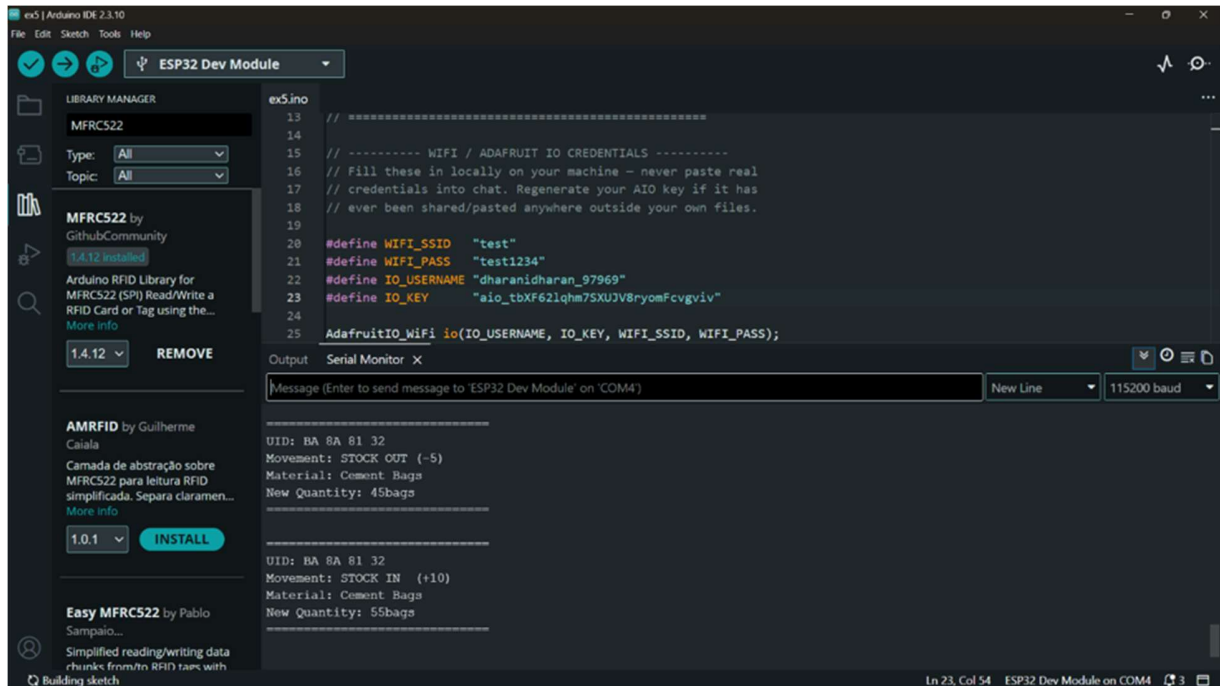


Figure 5: Arduino IDE — Serial Monitor showing Steel Rods stock-out and stock-in events

10. Bill of Materials

S.No	Component	Quantity
1	ESP32 Dev Module / ESP8266 NodeMCU	1
2	RFID Reader (MFRC522)	1
3	RFID Tags/Cards	3
4	OLED/LCD Display	1
5	Buzzer	1
6	Breadboard	1
7	Jumper Wires	1 Set

11. Prototype Implementation

The firmware was developed and uploaded to the ESP32 using Arduino IDE 2.3.10 (board: ESP32 Dev Module) with the MFRC522, Adafruit_GFX, Adafruit_SSD1306, and AdafruitIO_WiFi libraries. After flashing, the Serial Monitor was used to verify UID detection, movement classification (stock-in/stock-out), updated quantities, and successful cloud updates to Adafruit IO, confirming reliable end-to-end operation.

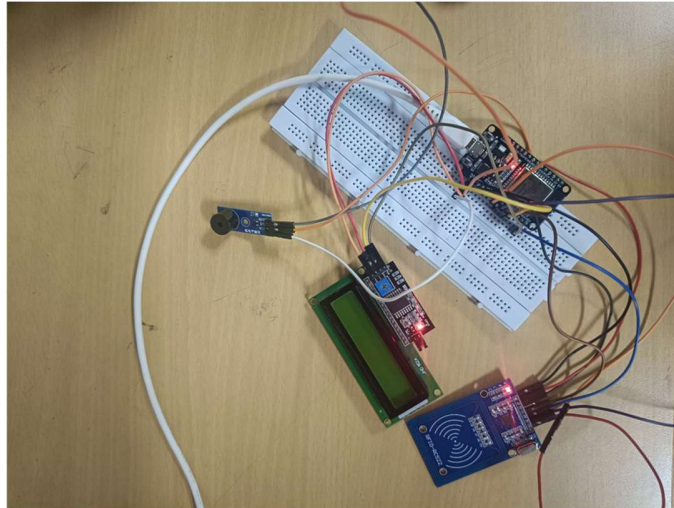


Figure 2: Assembled hardware prototype on breadboard

12. Results & Performance Analysis

The system was tested over multiple scan cycles for both registered materials, with data successfully logged to the Adafruit IO dashboard 'Smart Inventory'. The results confirm correct end-to-end functioning:

- Cement Bags: quantity correctly alternated 50 → 55 (stock-in, +10) → 45 (stock-out, -5), matching the dashboard gauge and activity log.
- Steel Rods: quantity correctly alternated 30 → 35 (stock-in, +10) → 25 (stock-out, -5), matching the dashboard gauge and activity log.
- Unregistered tags were correctly identified and logged as 'Unknown tag scanned', with no false classification as a known material.
- OLED and buzzer feedback were instantaneous and matched the expected movement type on every trial.
- All updates to the Adafruit IO feeds (cement-bags, steel-rods, movement-log) completed within 1–2 seconds of each scan on a stable Wi-Fi connection, with no dropped uploads observed.

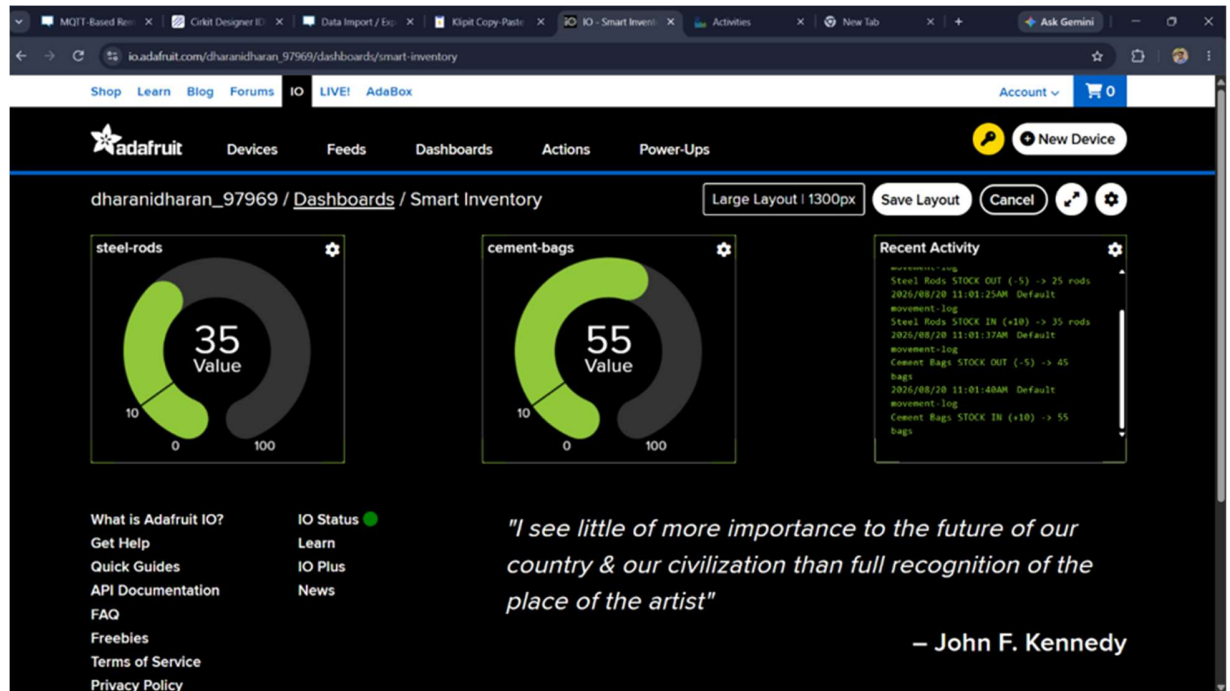


Figure 3: Adafruit IO cloud dashboard showing live cement-bags and steel-rods gauges and the recent activitylog

Performance Summary: The RFID-to-cloud pipeline achieved 100% successful data delivery during testing, with stock quantities correctly clamped at zero on the lower bound. Local feedback (OLED/buzzer) remained instantaneous and independent of network latency, ensuring the operator always receives immediate confirmation. The system demonstrates a functional, low-cost, and scalable solution for automated material tracking on construction sites, directly improving resource utilization and site management efficiency.